← Back to all posts

A Checklist for Turning a Private Plugin into a Recipe for Others

April 27, 2026

> I made something great, and now I want to share it with others!

This sentiment has been thought by 100s of plugin authors, willing to transform their work from private benefit to public good. To do that, authors go through our *recipe review* process. This comes in two steps:

- Automated feedback from CHEF, a tool to catch common mistakes or oversights before submission.
- Manual feedback from a human, where we look over everything to make sure it meets our quality bar, then provide you with specific feedback and guidance about changes, if

needed, that are required before publishing.

While it is ever evolving, here is the steps we go through to make sure your recipe is ready to go, broken down into sections of the plugin. Some quick terminology:

user: Someone who installs or forks a plugin recipe.
author: Creator and maintainer of a plugin recipe.

Plugin Ecosystem

Recipes are evaluated on how they contribute to the plugin ecosystem overall. This includes a few core ideals:

Collaboration over competition. A new recipe should not be seen by a user as indistinguishable from an existing recipe. If there is a similar plugin, it should add easily identifiable value.
Efforts to improve existing plugins should be the first priority, but we understand that if the original author is non-responsive or your suggestions are outside of the goals they have for their recipe, that a new recipe can be appropriate.
Similar topics, but different data sources, are great examples of where *competition* is valuable and completely acceptable.

Depth, not numbers. Variations on an idea should be encompassed within a single recipe. Whenever possible, a recipe that serves multiple themes, layouts, or even highlights different data *from the same data source* by using form fields is more valuable than multiple recipes creating *noise* in the search results. We consider *omnibus* plugins like Comic Library strong additions to the ecosystem.
TL;DR We want higher quality, not just more.

Family friendly. The default option is always family friendly and non-offensive. If content (language, images) that wouldn't be appropriate for a 13-year-old could appear, it should be **default** filtered *off*. If a recipe cannot be, you can publish it as **unlisted**, which makes it available to others, but will not show up in any search results.
Additionally, we have special categories (set in Form Fields), for things that are on the edge. For instance, the `morbid` category is searchable as a #category, but recipes are excluded from search results using normal terms.

Form Fields

Make sure the `author_bio` offers at least one way that a user could get in-touch with the author and that appropriate categories are chosen.

Check that `default` and `placeholder` is being used appropriately and there are no erroneous fields like `optional: false` (you only need to include it if it is true).

Update any plain-text links to use HTML `href` links and include `<br/>` when it was clear the author intended a line break.

Test each of the form fields to make sure the functionality offered is *actually* working. This involves making adjustments, saving, and looking within *edit markup* to see the changes. This also includes making use of `"key":"value"` pairs for options if it improves readability to the user and usability for the author. Also check that *truthy* values like *yes, true, 1* are evaluated properly in Liquid logic.

Validate no personal information will leak and that, if necessary, demo data is used.

Plugin Strategy

Polling

We use the *parse* feature to make sure the URL is processing logically and to understand if the endpoint is privately hosted or a public API. If public, but requiring authorization, that appropriate links and instructions are included in the Form Fields. Usually this isn't a source for problems, but more of an opportunity for improvement because Liquid logic is supported in this field.

We also will look to see if it is forcing "fresh data" by calling a URL that returns randomized data every time. If it's external, we recommend switching it to ours: `https://trmnl.com/custom_plugin_example_data.json` .

Webhook

A webhook requires extra steps by the user that need to be outlined or linked to in the `author_bio` section of the Form Fields. Most commonly this is to a Github project page which has a detailed ReadMe (which we'll review for clarity and may even try).

Static

Rarely requires review unless other issues require us taking a look.

Plugin Merge

Make use of the `plugin_instance_select` form field to help the user select the proper plugin easily and that instructions are included in `author_bio` 's description.

Markup

For maximum compatibility, we require recipes authors to prioritize using our design Framework and Liquid markup for logic first, only followed by CSS and Javascript when these reach their limit. Javascript-driven recipes can be unique and amazing when a recipe goes beyond text and images, so we don't want discourage that creativity.

Red Flags

These are items that will cause problems.

`async` API calls. Renderer will not wait more than 5 seconds; consider using Serverless or your own hosted middleware. 99% of the time, any API calls should be in your POLLING URLs section (you can have multiple URLs, one on each line).

Charting without using a unique class identifier.

Javascript that is waiting for page load rather than the event listener DOMContentLoaded.

Using the inline CSS property `opacity` , which gives a false sense of "working" in preview, but will not work well when rendered in 1-bit (black and white). Instead, use Framework classes like `text--black 2bit:text--gray-30 4bit:text--gray-20` to target grayscale.

Custom fonts. For certain aesthetics, fine, but 9 out of 10 times you should stick to the fonts we provide and use for different Framework elements automatically. This ensures great legibility at different sizes.

Layouts

We preview every *view* (e.g. Full, Quadrant) across both TRMNL OG and TRMNL X in *landscape* mode, as well as TRMNL X in *portrait* mode, to verify the screen is **mostly content, not whitespace**. We also check if any data is getting cutoff horizontally or pushed off vertically unintentionally.

Most recipes that fail at larger sizes haven't taken into account our responsive classes (e.g. `lg:` or `portrait:`) as outlined in our TRMNL X Framework Guide or are not using Framework.

Here are some of our favorite Framework tricks for great layouts.

Content Overflow

Make use of automatic length limiting with:

- Variable level Liquid truncate
- Line level data clamping
- View level overflow, including table overflow

Charts

Rather than using a fixed `height` for your chart, setting height to `null` will let it fill its container, so you can control it using your other markup.

Using Liquid Templates

Liquid templates offer a great way to re-use content. However, any data used in the template needs to be passed *into* the template. This means that the `trmnl` native variables need to be passed in as well if you are using them in your template. For example: `{% render "my_template", trmnl: {{ trmnl }} %}`.

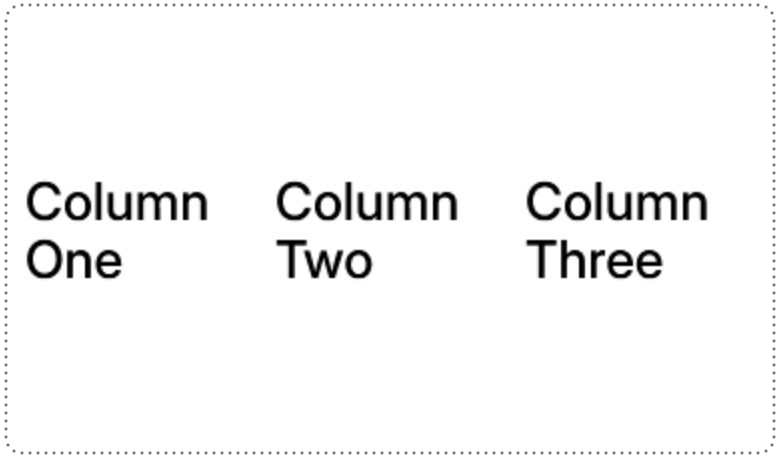
Flexibly Rearranging Content

Adjust the `layout` or `flex` direction in portrait mode. For instance, `flex flex--row portrait:flex--col` can quickly reflow your content. You can even stack responsive classes `lg:portrait` as noted in our docs.

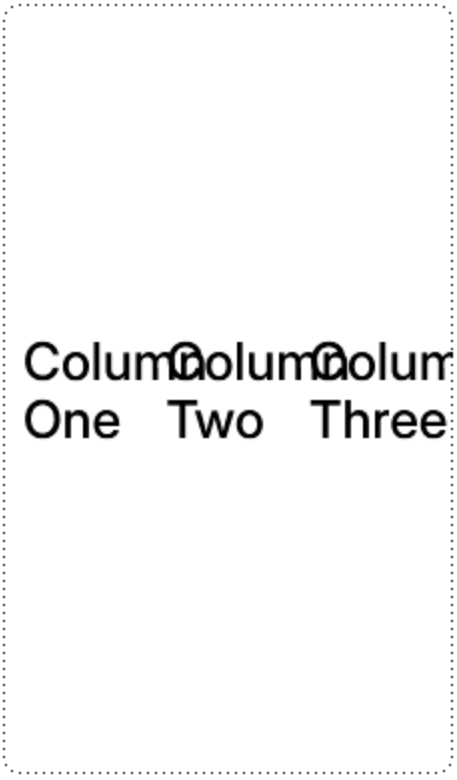
Using `grid` and the `grid--cols-#` feature to reflow content. Here is an example:

```
<div class="layout">
  <div class="grid grid--cols-3">
    <span class="value value--small lg:value--base">Column One</span>
    <span class="value value--small lg:value--base">Column One</span>
    <span class="value value--small lg:value--base">Column One</span>
  </div>
</div>
```

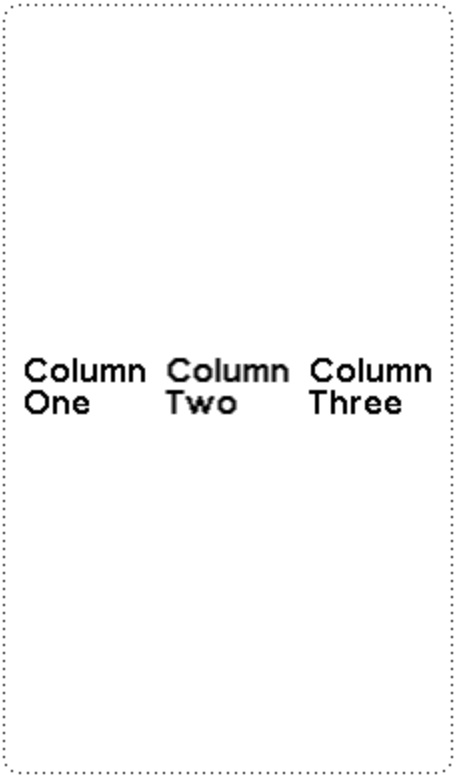
Landscape mode looks fine.



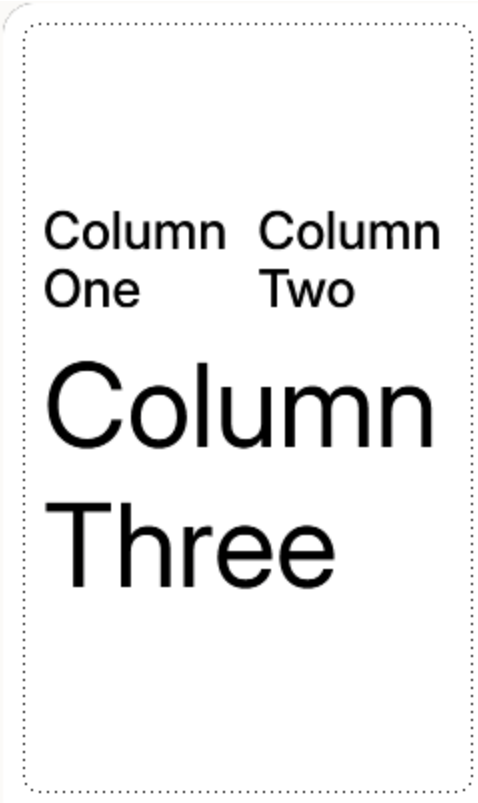
Switch to portrait and we have collisions.



A quick fix could be to shrink the text in portrait mode using `portrait:value--xxsmall`, but that increases the whitespace dramatically.



But instead by using `portrait:grid--cols-2` we get to actually increase the size and reduce the whitespace.




```
<div class="layout">
  <div class="grid grid--cols-3 portrait:grid--cols-2">
```

```
    <span class="value value--small lg:value--base">Column One</span>
    <span class="value value--small lg:value--base">Column Two</span>
    <span class="value value--small portrait:col--span-2 portrait:value--large lg:value--b
  </div>
</div>
```

The Little Things

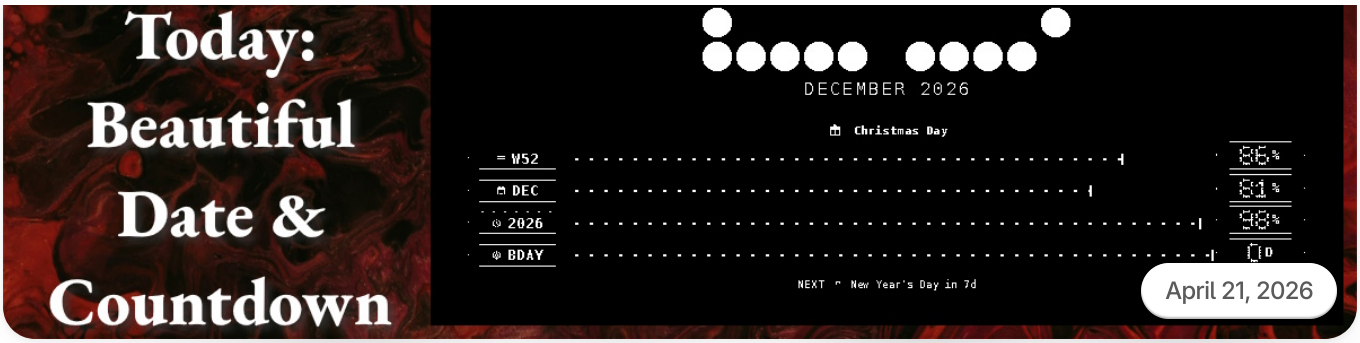
We don't always notice these, and mostly they are nice-to-haves and not required for publishing.

- If you use a `title_bar`, don't use the default TRMNL logo that is in our example docs, but instead use an image appropriate for the data source. Be careful with emoji, they often don't render as expected.
- *Using a static image inserted via a `src` URL?* If you want to make sure that it always renders, you can encode it within the page rather than calling an external URL. Use our guide to inline images.
- Keep things **DRY** [Don't Repeat Yourself]. The *Shared* view is a great way to accomplish this.

Mario Lurig

Developer Relations Manager

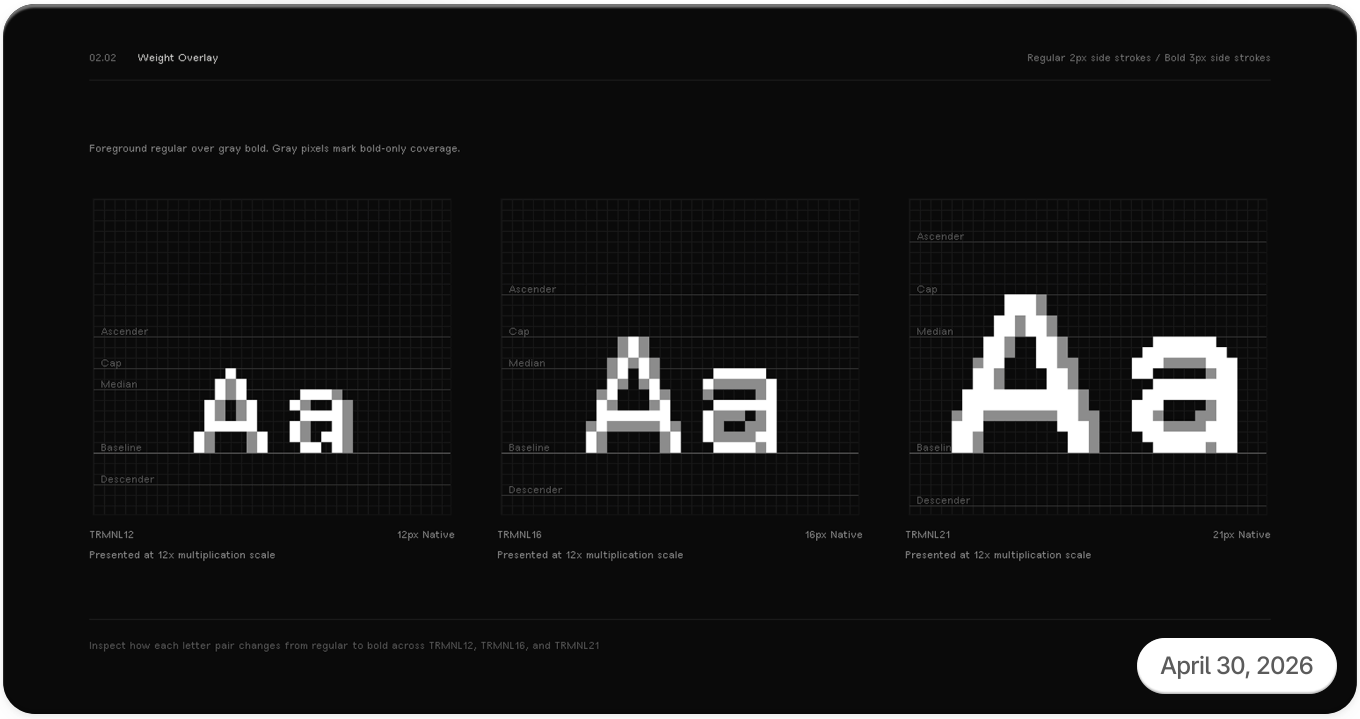
‹ Previous



Featured Community Plugin - Today: Beautiful Date & Countdown

A countdown is anticipation manifested visually, and today's beautiful countdown featured plugin gives you a rich, yet minimal view of your target date.

Next ›



Introducing Typography

On designing a font for ePaper displays.

Engineered on Earth Assembled in Atlanta & Berlin

TRMNL PLUGINS

Home Page
About Us
Blog Posts
Brand Resources

Integrations
Recipes

DEVELOPERS

For Developers
Framework
Create Plugins
Tailor

DOCUMENTATION

Get Started
BYODevice
BYOServer
BYODevice/Server
Public API
Partners API

BUSINESS

For Business
Case Studies

SUPPORT

Support
Status
Privacy
Terms